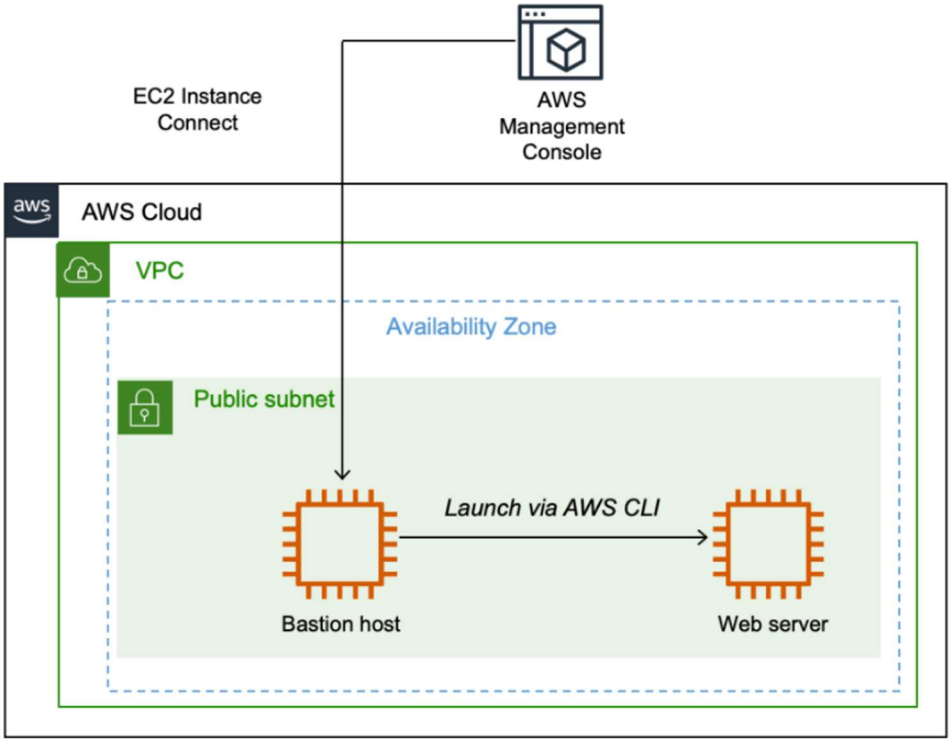
171--Lab - Creación de instancias de Amazon EC2

En esta práctica se muestra como lanzar una instancia como host de bastión y, desde la línea de comandos de AWS en esta instancia, lanzar otra instancia que funcionará como servidor web.

Arquitectura

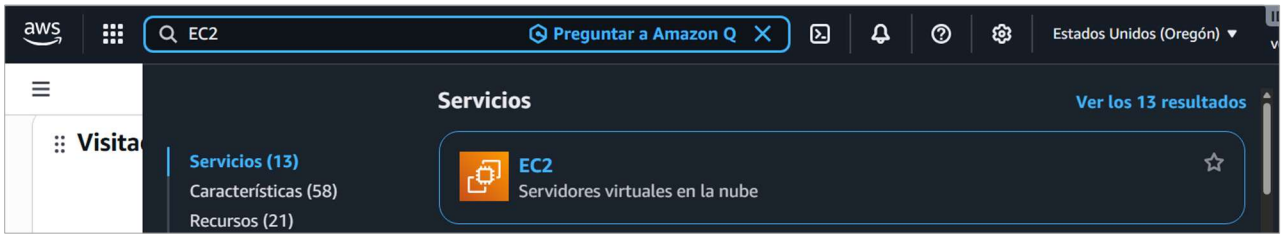
La siguiente es la arquitectura a implementar:



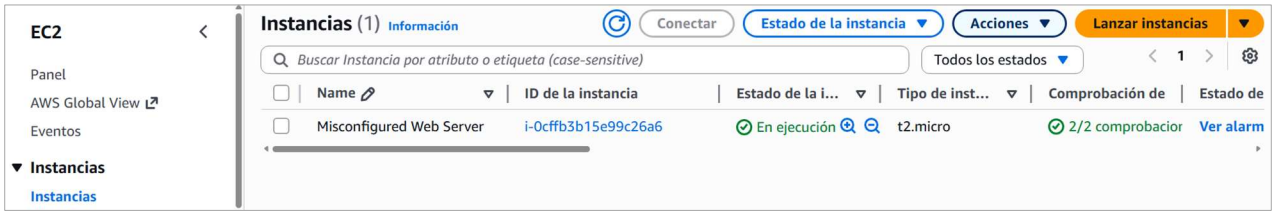
Lanzar una instancia EC2 mediante la consola de administración de AWS

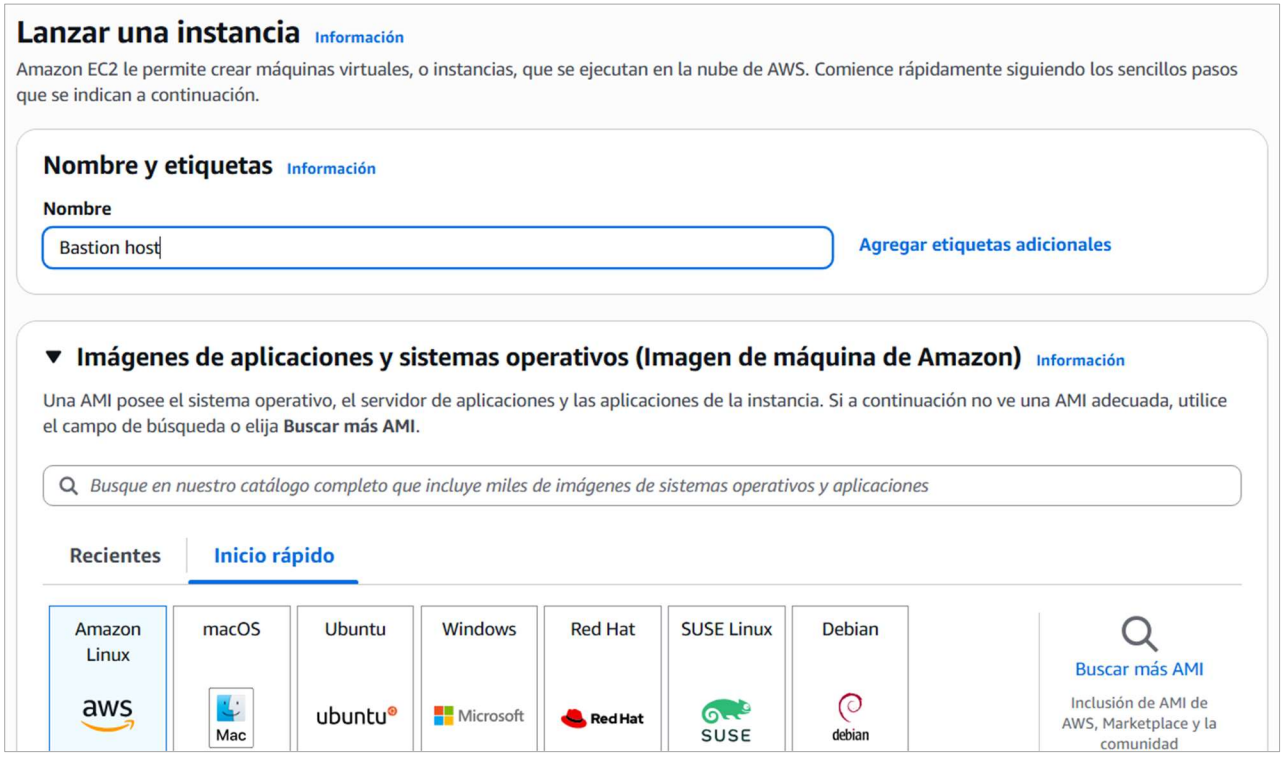
Un bastion host (también conocido como "servidor de salto" o jump server) es un servidor especializado diseñado para actuar como el único punto de entrada seguro desde internet hacia una red privada.

Para crear un bastion host accedemos al panel de instancias

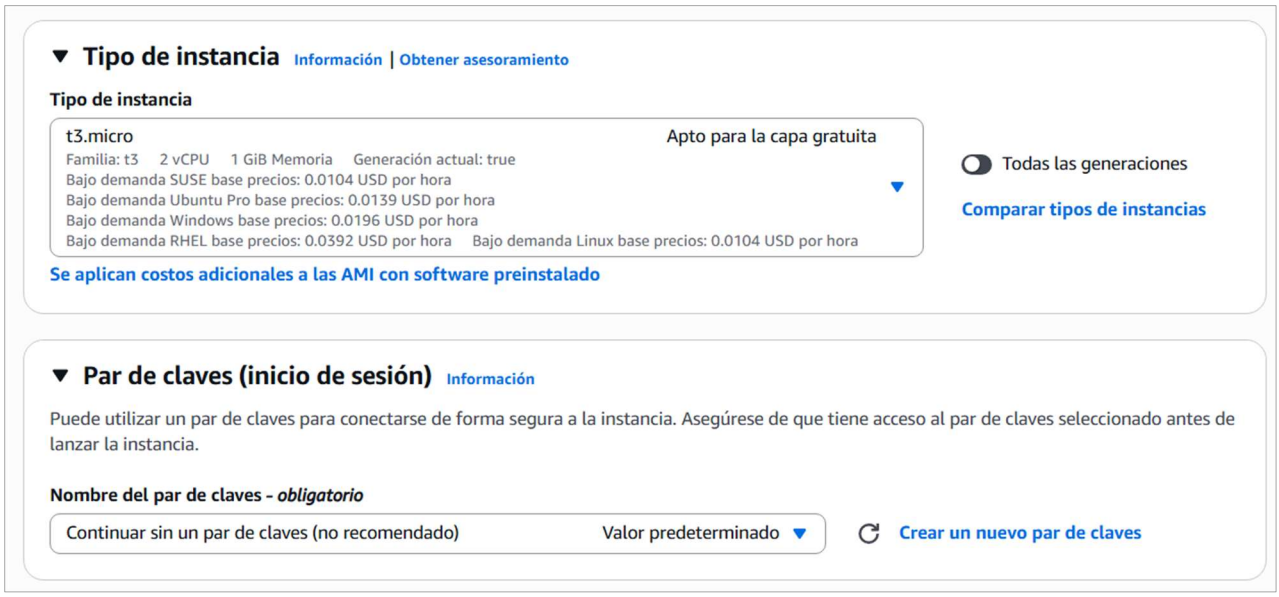


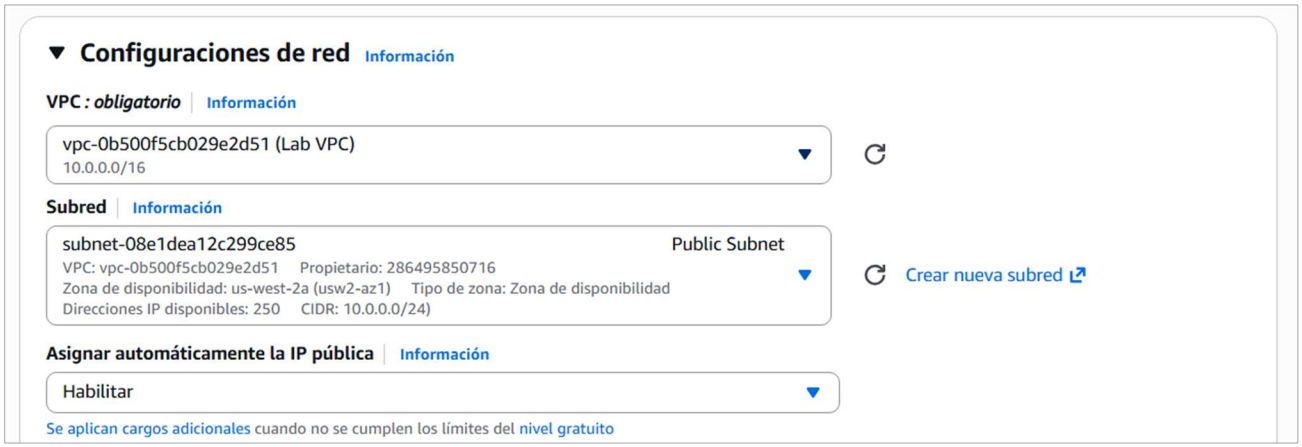
Hacemos clic en *Lanzar instancias*



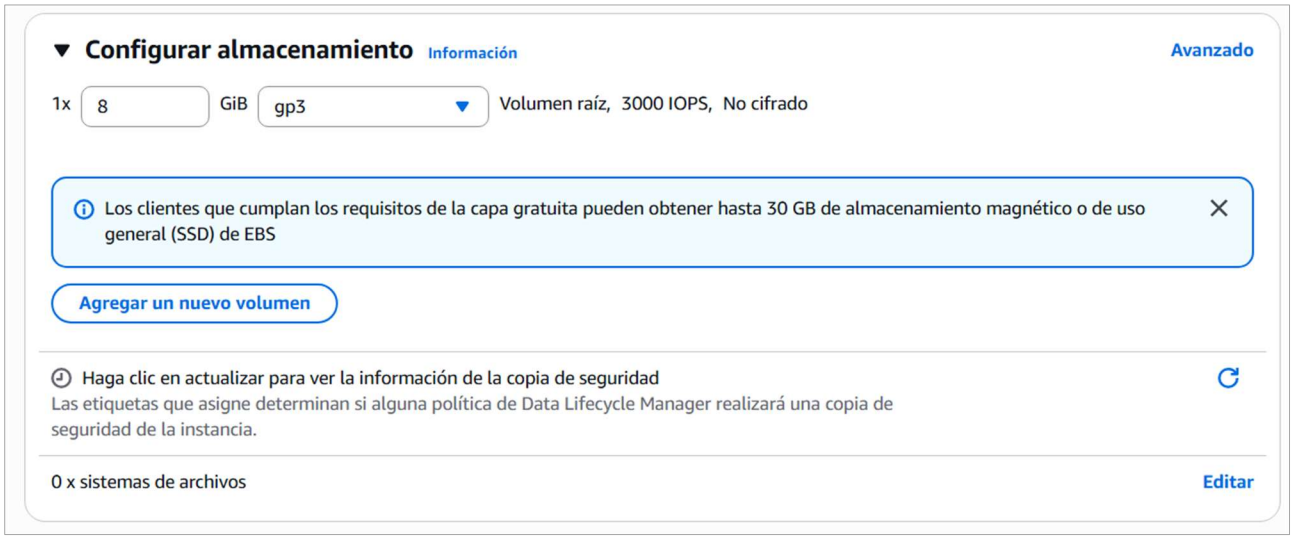


Especificamos que no usaremos un par de claves porque nos conectaremos mediante *EC2 Instance Connect*





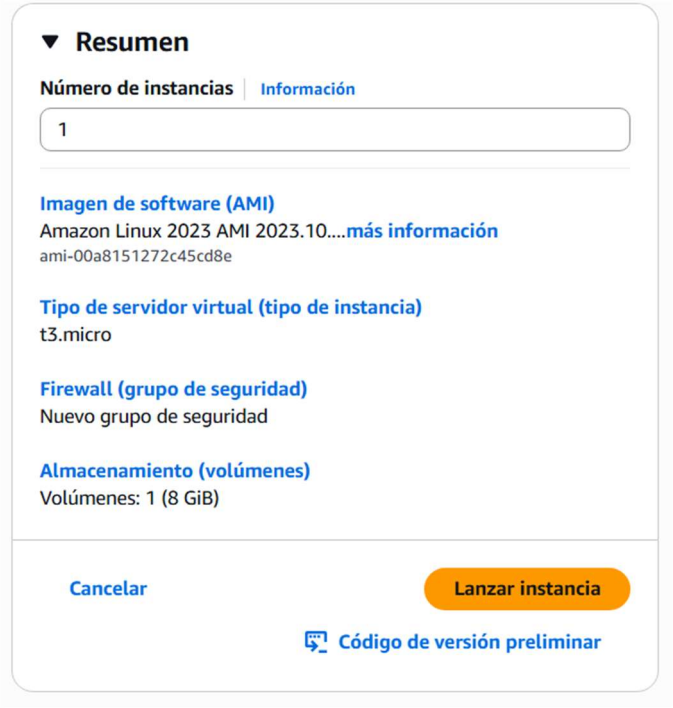
Creamos un grupo de seguridad y nos aseguramos que tenga una regla de entrada que permita el acceso por SSH, de este modo podremos conectarnos mediante *EC2 Instance Connect*



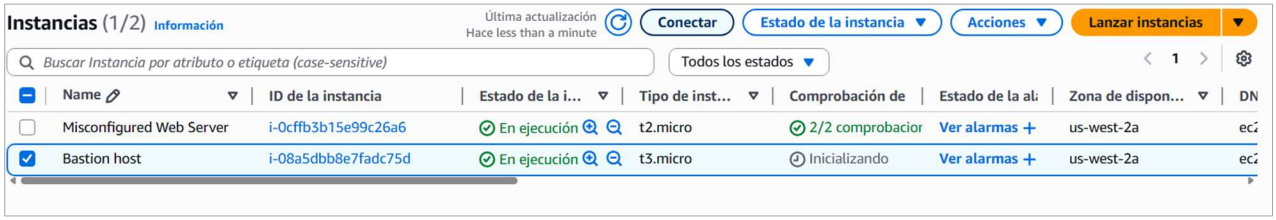
Para un bastion host requerimos seleccionar *Bastion-Role* para el perfil de instancia de IAM.

Este rol permite que el servidor pueda hablar con otros servicios de AWS, como S3 o IAM, de manera segura. Esto es requerido porque desde el AWS CLI nos comunicaremos con el servicio de Amazon EC2 para lanzar otra instancia.

Lanzamos la instancia

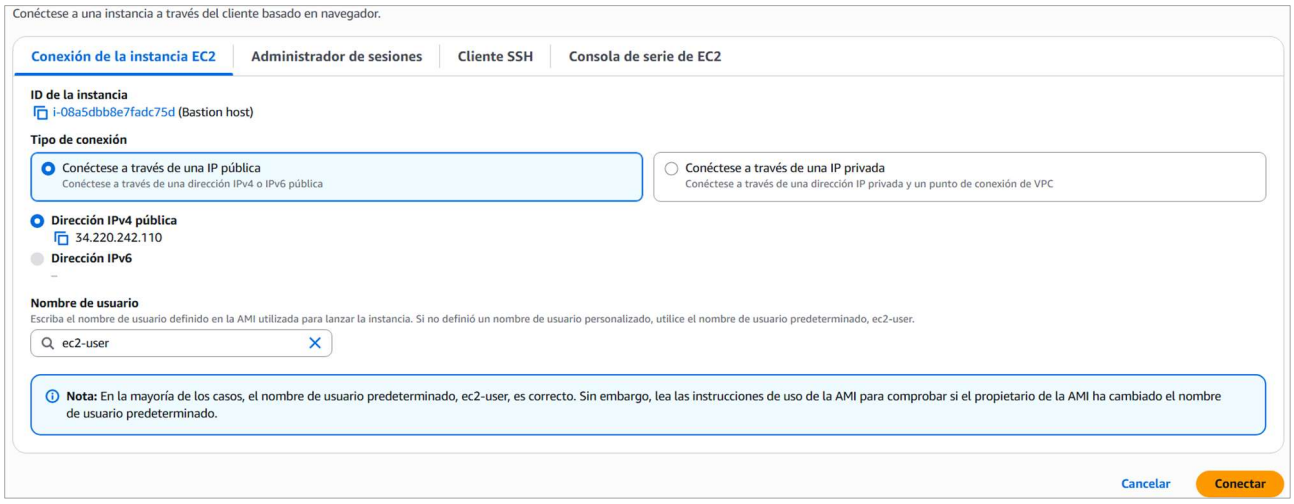


La instancia está en ejecución



Conectar al bastion host

Nos conectamos al bastion host mediante *EC2 Instance Connect*



Lanzar una instancia EC2 mediante el AWS CLI

Con el siguiente comando indicamos al CLI la región con la que estamos trabajando

```
#Set the Region
AZ=`curl -s http://169.254.169.254/latest/meta-data/placement/availability-zone`
export AWS_DEFAULT_REGION=${AZ::-1}
```

AWS continuamente aplica parches y actualiza las AMIs, por lo tanto se recomienda siempre usar la última versión de las AMIs al lanzar instancias.

En el siguiente comando utilizamos *AWS Systems Manager Parameter Store* para obtener el ID de la AMI más reciente de Amazon Linux 2. AWS mantiene una lista de AMIs en *Parameter Store*.

```
#Retrieve latest Linux AMI
AMI=$(aws ssm get-parameters --names /aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2 --query 'Parameters[0].[Value]' --output text)
echo $AMI
```

- --query 'Parameters[0].[Value]': filtra la respuesta para quedarse únicamente con el ID de la imagen (el valor), descartando el resto de la información técnica

- --output text: asegura que el resultado sea texto plano para que pueda guardarse limpiamente en la variable

```
[ec2-user@ip-10-0-0-196 ~]$ #Set the Region
AZ=`curl -s http://169.254.169.254/latest/meta-data/placement/availability-zone`
export AWS_DEFAULT_REGION=${AZ::-1}
#Retrieve latest Linux AMI
AMI=$(aws ssm get-parameters --names /aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2 --query 'Parameters[0].[Value]' --output text)
echo $AMI
-bash: -1: substring expression < 0
ami-0c0ba9105fe7293eb
```

Ahora obtenemos el id de la subred con el nombre "Public Subnet"

```
SUBNET=$(aws ec2 describe-subnets --filters 'Name=tag:Name,Values=Public Subnet' --query Subnets[].SubnetId --output text)
echo $SUBNET
```

```
[ec2-user@ip-10-0-0-196 ~]$ SUBNET=$(aws ec2 describe-subnets --filters 'Name=tag:Name,Values=Public Subnet' --query Subnets[].SubnetId --output text)
echo $SUBNET
subnet-08e1dea12c299ce85
```

Obtenemos el id del grupo de seguridad con el nombre "WebSecurityGroup"

```
SG=$(aws ec2 describe-security-groups --filters Name=group-name,Values=WebSecurityGroup --query SecurityGroups[].GroupId --output text)
echo $SG
```

```
[ec2-user@ip-10-0-0-196 ~]$ SG=$(aws ec2 describe-security-groups --filters Name=group-name,Values=WebSecurityGroup --query SecurityGroups[].
GroupId --output text)      echo $SG
sg-08de40dc4b58c76b6
```

Descargamos el script que incluiremos en los datos de usuario de la instancia

```
wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-100-RSJAWS-1-23732/171-lab-JAWS-create-ec2/s3/UserData.txt
```

```
[ec2-user@ip-10-0-0-196 ~]$ wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-100-RSJAWS-1-23732/171-lab-JAWS-create-ec2/s3/U
serData.txt                 wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-100-RSJAWS-1-23732/171-lab-JAWS-create-ec2/s3/Us
erData.txt
--2026-01-20 05:23:42--  https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-100-RSJAWS-1-23732/171-lab-JAWS-create-ec2/s3/UserData.
txt
Resolving aws-tc-largeobjects.s3.us-west-2.amazonaws.com (aws-tc-largeobjects.s3.us-west-2.amazonaws.com)... 3.5.86.117, 52.92.213.42, 52.218.
180.33, ...
Connecting to aws-tc-largeobjects.s3.us-west-2.amazonaws.com (aws-tc-largeobjects.s3.us-west-2.amazonaws.com)|3.5.86.117|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 327 [text/plain]
Saving to: 'UserData.txt'

UserData.txt                                  0%[                                                                                    ]       U
serData.txt                                 100%[===================================================================================>]     327
  --.-KB/s    in 0s

2026-01-20 05:23:42 (14.9 MB/s) - 'UserData.txt' saved [327/327]
```

El contenido del script es para instalar el servidor Apache, habilitar y activar el servicio y descargar los archivos de la aplicación

```
[ec2-user@ip-10-0-0-196 ~]$ cat UserData.txt
#!/bin/bash
# Install Apache Web Server
yum install -y httpd

# Turn on web server
systemctl enable httpd.service
systemctl start  httpd.service

# Download App files
wget https://aws-tc-largeobjects.s3.amazonaws.com/CUR-TF-100-RESTRT-1/171-lab-%5BJAWS%5D-create-ec2/dashboard-app.zip
unzip dashboard-app.zip -d /var/www/html/
```

Ahora lanzamos la nueva instancia utilizando las variables que creamos anteriormente

```
INSTANCE=$(\
aws ec2 run-instances \
--image-id $AMI \
--subnet-id $SUBNET \
--security-group-ids $SG \
--user-data file:///home/ec2-user/UserData.txt \
--instance-type t3.micro \
--tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=Web
Server}]' \
--query 'Instances[*].InstanceId' \
--output text \
)
echo $INSTANCE
```

Se le ha asignado el nombre "Web Server"

El parámetro –query especifica que devuelva el ID de la instancia

```
[ec2-user@ip-10-0-0-196 ~]$ INSTANCE=$(\
aws ec2 run-instances \
--image-id $AMI \
--subnet-id $SUBNET \
--security-group-ids $SG \
--user-data file:///home/ec2-user/UserData.txt \
--instance-type t3.micro \
--tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=Web Server}]' \
--query 'Instances[*].InstanceId' \
--output text \
)
echo $INSTANCE
i-064f12918049173e6
```

El ID de la instancia se ha guardado en la variable $INSTANCE

Ahora mostramos el estado de la instancia

```
aws ec2 describe-instances --instance-ids $INSTANCE --query
'Reservations[].Instances[].State.Name' --output text
```

```
[ec2-user@ip-10-0-0-196 ~]$ aws ec2 describe-instances --instance-ids $INSTANCE --query 'Reservations[].Instances[].State.Name' --output text
running
```

La instancia está en ejecución

Para mostrar la metadata de la instancia ejecutamos el siguiente comando

```
aws ec2 describe-instances --instance-ids $INSTANCE
```

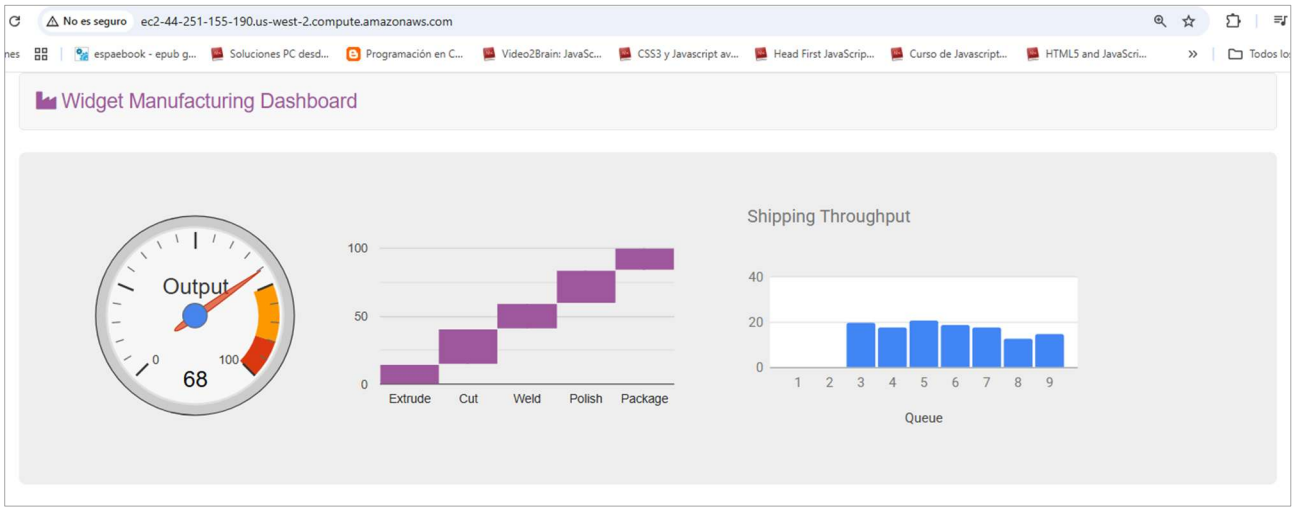
```
[ec2-user@ip-10-0-0-196 ~]$ aws ec2 describe-instances --instance-ids $INSTANCE
{
    "Reservations": [
        {
            "ReservationId": "r-0970d6d9d26da9c62",
            "OwnerId": "286495850716",
            "Groups": [],
            "Instances": [
                {
                    "Architecture": "x86_64",
                    "BlockDeviceMappings": [
                        {
                            "DeviceName": "/dev/xvda",
                            "Ebs": {
                                "AttachTime": "2026-01-20T05:26:07+00:00",
                                "DeleteOnTermination": true,
                                "Status": "attached",
                                "VolumeId": "vol-0a2d3f95e159dc840"
                            }
                        }
                    ],
                    "ClientToken": "126d7d29-e67d-409f-98ab-1f12cc12e1bd",
                    "EbsOptimized": false,
                    "EnaSupport": true,
```

Para mostrar la dirección IPv4 pública de la instancia ejecutamos el siguiente comando

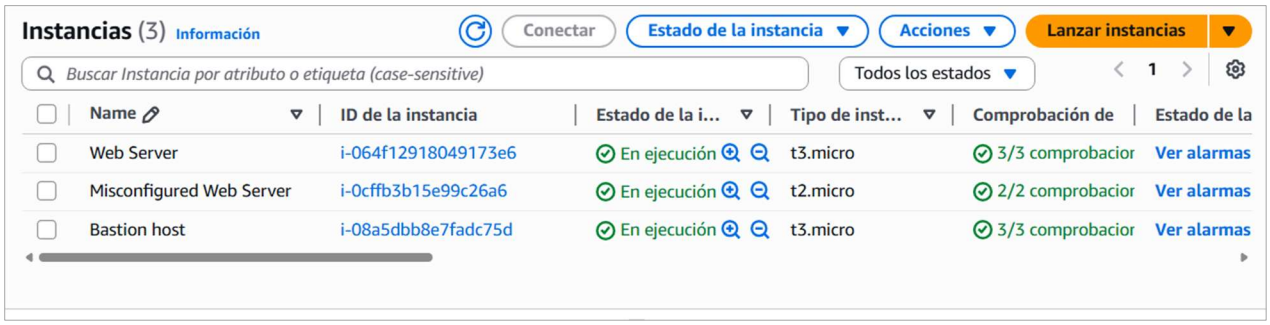
```
aws ec2 describe-instances --instance-ids $INSTANCE --query
Reservations[].Instances[].PublicDnsName --output text
```

```
[ec2-user@ip-10-0-0-196 ~]$ aws ec2 describe-instances --instance-ids $INSTANCE --query Reservations[].Instances[].PublicDnsName --output text
ec2-44-251-155-190.us-west-2.compute.amazonaws.com
```

Accedemos a la dirección IPv4 pública desde el navegador y podemos ver la aplicación

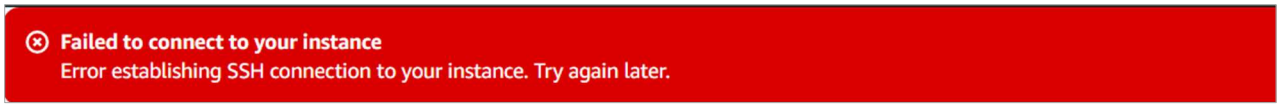


Desde la consola de administración de AWS también podemos ver la instancia

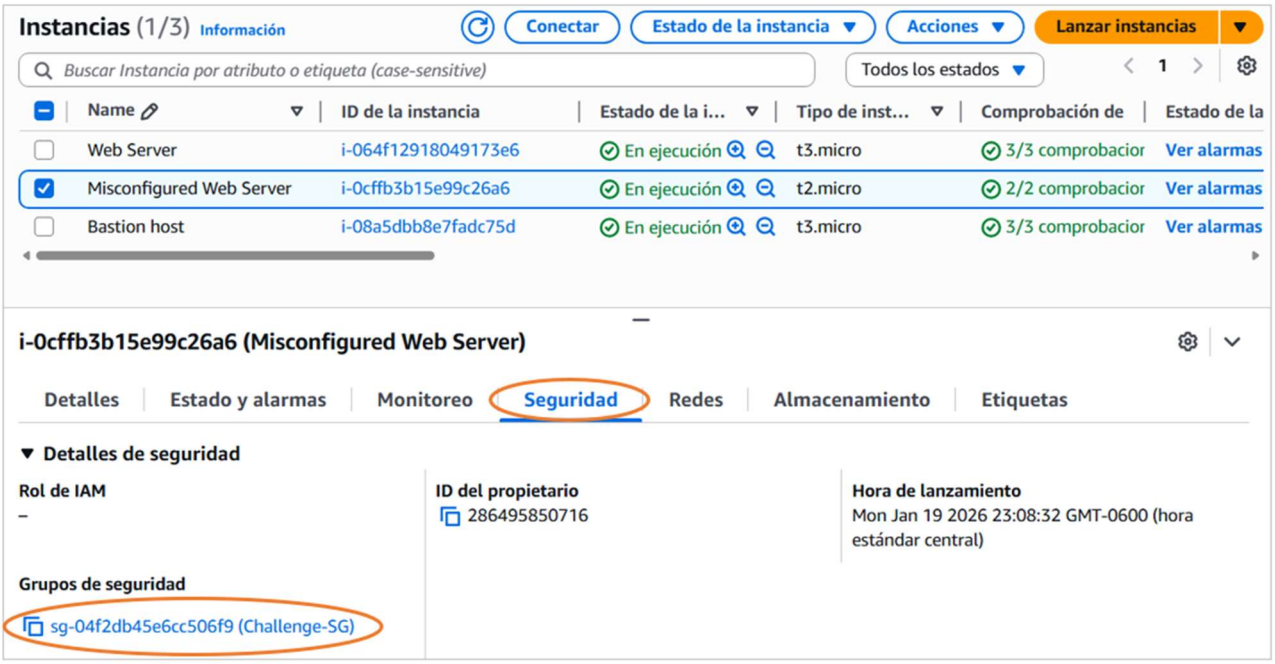


Desafío 1: Conectar a una instancia EC2

Si intentamos conectarnos a la instancia "Misconfigured Web Server" mediante *EC2 Instance Connect* muestra el siguiente mensaje de error:



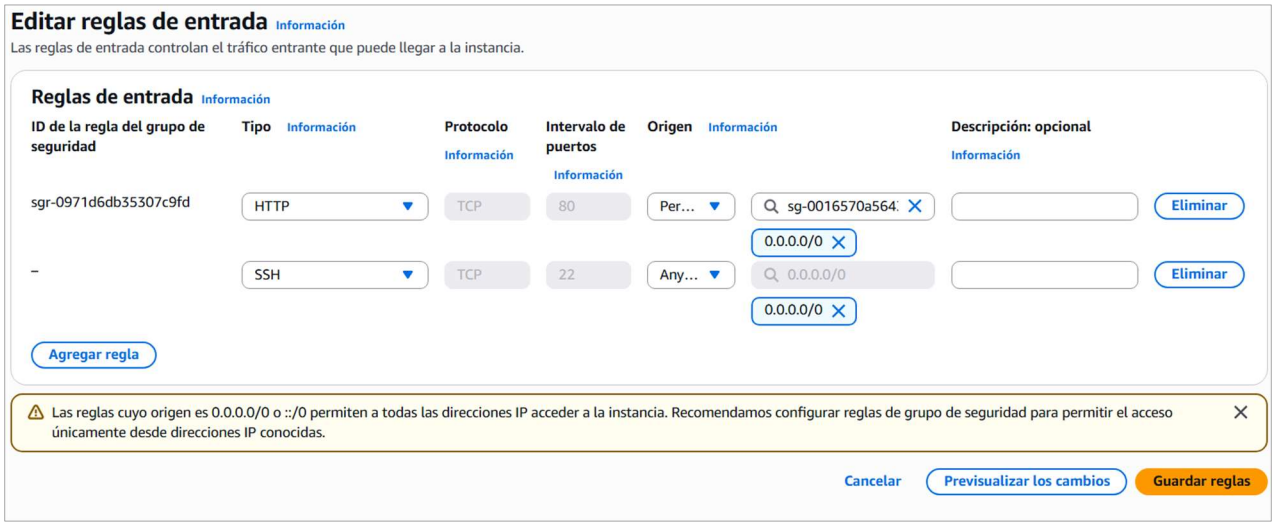
Para verificar la causa del error checamos el grupo de seguridad de la instancia



Podemos ver que en el grupo de seguridad no hay una regla de entrada que permita el acceso por SSH. Por tal razón no es posible conectarse desde *EC2 Instance Connect*.

Hacemos clic en *Editar reglas de entrada*

Agregamos el acceso por SSH



Ahora la conexión es exitosa

```
   ,     #_
   ~\_  ####_        Amazon Linux 2
  ~~  \_#####\
  ~~     \###|       AL2 End of Life is 2026-06-30.
  ~~       \#/ ___
   ~~       V~' '->
    ~~~         /    A newer version of Amazon Linux is available!
      ~~._.   _/
         _/ _/       Amazon Linux 2023, GA and supported until 2028-03-15.
       _/m/'           https://aws.amazon.com/linux/amazon-linux-2023/

[ec2-user@ip-10-0-0-138 ~]$
```

i-0cffb3b15e99c26a6 (Misconfigured Web Server)

PublicIPs: 35.91.151.51 PrivateIPs: 10.0.0.138

Desafío 2: Reparar la instalación del servidor web

Si intentamos acceder a la dirección IPv4 pública de la instancia "Misconfigured Web Server" muestra un mensaje que indica que la página no se ha encontrado.

Desde *EC2 Instance Connect* verificamos el estado del servidor Apache.

El servidor Apache está inactivo

```
[ec2-user@ip-10-0-0-138 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: disabled)
   Active: inactive (dead)
     Docs: man:httpd.service(8)
[ec2-user@ip-10-0-0-138 ~]$
```

i-0cffb3b15e99c26a6 (Misconfigured Web Server)

PublicIPs: 35.91.151.51 PrivateIPs: 10.0.0.138

Activamos y habilitamos el servidor Apache

```
[ec2-user@ip-10-0-0-138 ~]$ sudo chkconfig httpd on
Note: Forwarding request to 'systemctl enable httpd.service'.
[ec2-user@ip-10-0-0-138 ~]$ sudo service httpd start
Redirecting to /bin/systemctl start httpd.service
[ec2-user@ip-10-0-0-138 ~]$
```

i-0cffb3b15e99c26a6 (Misconfigured Web Server)

PublicIPs: 35.91.151.51 PrivateIPs: 10.0.0.138

Checamos nuevamente el estado

El servidor ahora está activo

```
[ec2-user@ip-10-0-0-138 ~]$ sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; vendor preset: disabled)
   Active: active (running) since Tue 2026-01-20 05:56:52 UTC; 1min 10s ago
     Docs: man:httpd.service(8)
 Main PID: 4110 (httpd)
   Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
   CGroup: /system.slice/httpd.service
           ├─4110 /usr/sbin/httpd -DFOREGROUND
           ├─4111 /usr/sbin/httpd -DFOREGROUND
           ├─4112 /usr/sbin/httpd -DFOREGROUND
           ├─4113 /usr/sbin/httpd -DFOREGROUND
           ├─4114 /usr/sbin/httpd -DFOREGROUND
           └─4115 /usr/sbin/httpd -DFOREGROUND

Jan 20 05:56:52 ip-10-0-0-138.us-west-2.compute.internal systemd[1]: Starting The Apache HTTP Server...
Jan 20 05:56:52 ip-10-0-0-138.us-west-2.compute.internal systemd[1]: Started The Apache HTTP Server.
[ec2-user@ip-10-0-0-138 ~]$
```

i-0cffb3b15e99c26a6 (Misconfigured Web Server)

PublicIPs: 35.91.151.51 PrivateIPs: 10.0.0.138

Verificamos nuevamente el acceso desde el navegador.

La página se muestra correctamente

No es seguro 35.91.151.51

Test Page

This page is used to test the proper operation of the Apache HTTP server after it has been installed. If you can read this page, it means that the Apache HTTP server installed at this site is working properly.

If you are a member of the general public:

The fact that you are seeing this page indicates that the website you just visited is either experiencing problems, or is undergoing routine maintenance.

If you would like to let the administrators of this website know that you've seen this page instead of the page you expected, you should send them e-mail. In general, mail sent to the name "webmaster" and directed to the website's domain should reach the appropriate person.

For example, if you experienced problems while visiting www.example.com, you should send e-mail to "webmaster@example.com".

If you are the website administrator:

You may now add content to the directory `/var/www/html/`. Note that until you do so, people visiting your website will see this page, and not your content. To prevent this page from ever being used, follow the instructions in the file `/etc/httpd/conf.d/welcome.conf`.

You are free to use the image below on web sites powered by the Apache HTTP Server:

Powered by APACHE 2.4

Nota: La razón por la que no requerimos usar *aws configure* en el Bastion Host es porque esa instancia tiene asignado un IAM Role (específicamente el Bastion-Role) que le otorga permisos automáticos.